Aim:

1.Importing the libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import micropip
await micropip.install(['seaborn'])
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
Matplotlib is building the font cache; this may take a moment.
```

2.Creating a random dataset and displaying the first 5 information.

```
np.random.seed(10)
data = {'Study_Hours' : np.random.normal(5,2,100).round(1),
        'Sleep_Hours' : np.random.normal(6,1.5,100).round(1),
        'Internet_Hours' : np.random.normal(3,1.3,100).round(1),
        'Class_Attendance' : np.random.normal(50,100,100).round(1)}
df = pd.DataFrame(data)
df['Passed'] = np.where((df['Study_Hours']*10 + df['Class_Attendance']) > 105, 1, 0)
noise_idx = np.random.choice(df.index, size = 10, replace = False)
df.loc[noise_idx, 'Passed'] = 1 - df.loc[noise_idx, 'Passed']
df.head()
```

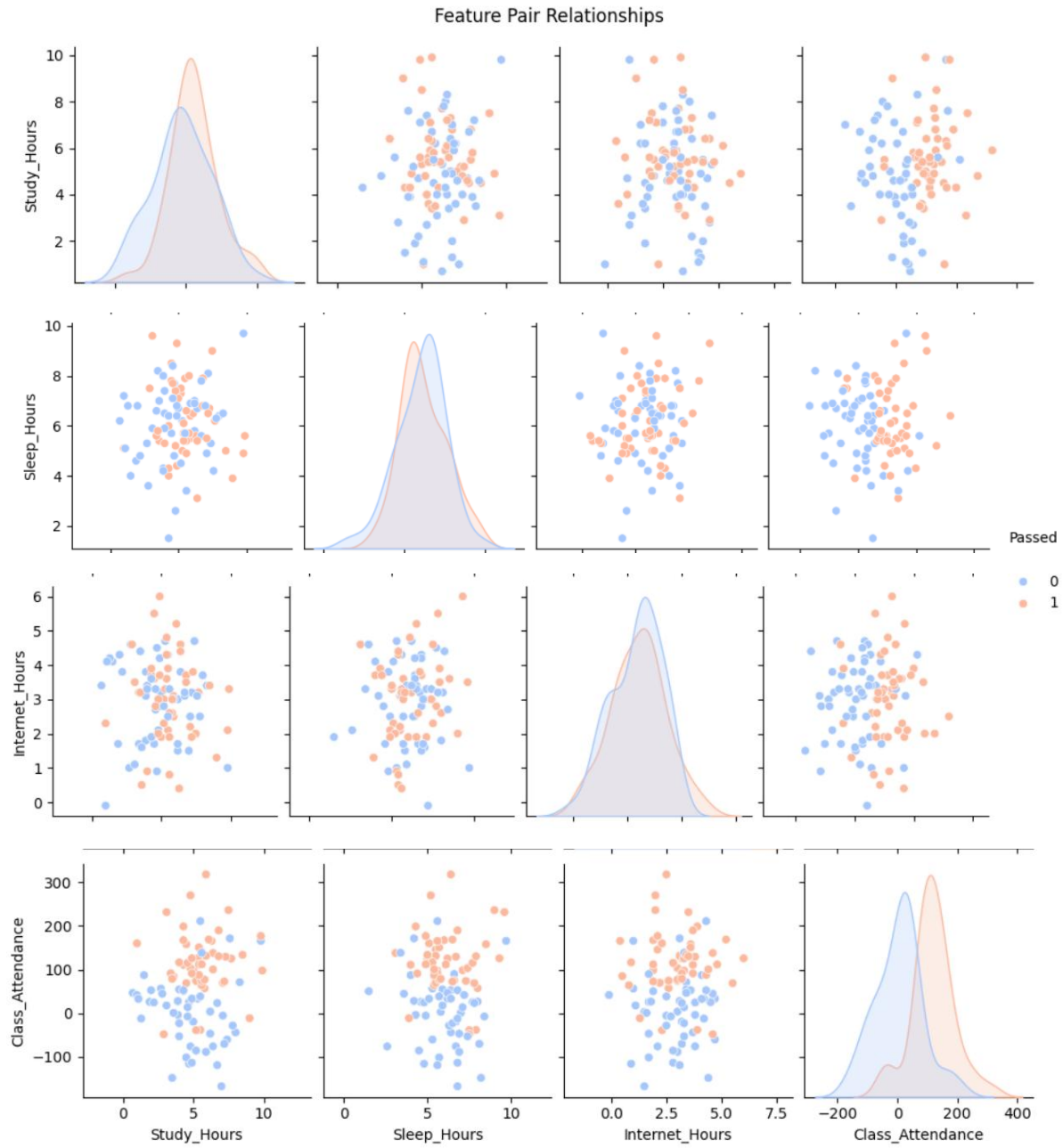
OUTPUT:

	Study_Hours	Sleep_Hours	Internet_Hours	Class_Attendance	Passed
0	7.7	6.2	3.2	125.5	1
1	6.4	3.1	4.6	137.9	1
2	1.9	4.6	1.7	26.0	0
3	5.0	6.7	3.2	-19.5	0
4	6.2	5.8	1.5	2.9	0

3.Visualizing the feature pair relationships.

```
import seaborn as sns
sns.pairplot(df, hue = 'Passed', palette = 'coolwarm')
plt.suptitle('Feature Pair Relationships', y=1.02)
plt.show()
```

OUTPUT:



4.Dropping the Passed column, so that we can predict the outcome.

```
X = df.drop('Passed', axis = 1)
y = df['Passed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

5.Intializing the model.

```
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier()
```

6.Checking the model accuracy, confusion matrix and classification report.

```
y_pred = model.predict(X_test)
print("Accuracy:" , accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

```
Accuracy: 0.8
Confusion Matrix:
 [[12  2]
 [ 2  4]]
Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.86      0.86        14
           1       0.67      0.67      0.67         6

    accuracy                           0.80        20
   macro avg       0.76      0.76      0.76        20
weighted avg       0.80      0.80      0.80        20
```

7.Checking the importance of each feature through visualization.

CODE:

```
importances = model.feature_importances_
features = X.columns
plt.figure(figsize = (8,5))
sns.barplot(x = importances, y = features)
plt.title("Feature Importance")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()
```

OUTPUT: